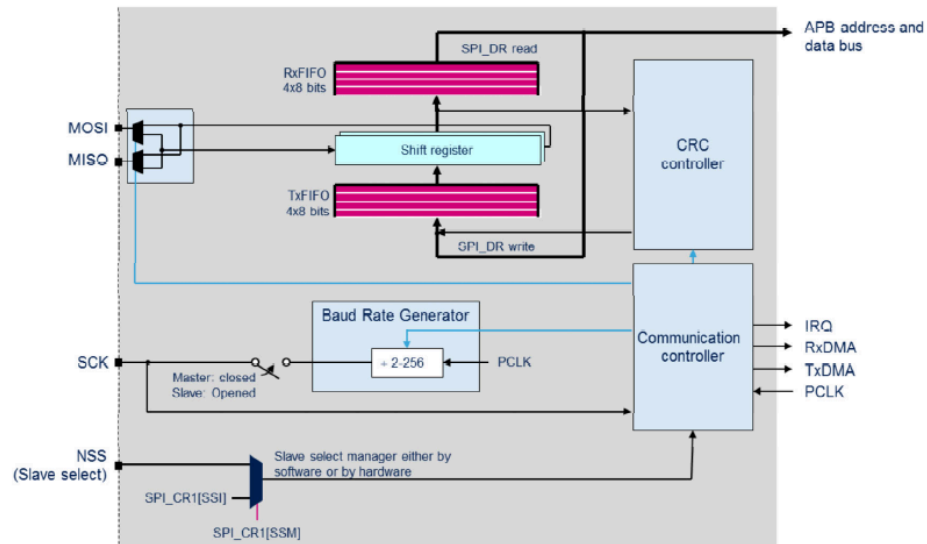


SPI (Serial Peripheral Interface)

Block diagram

5

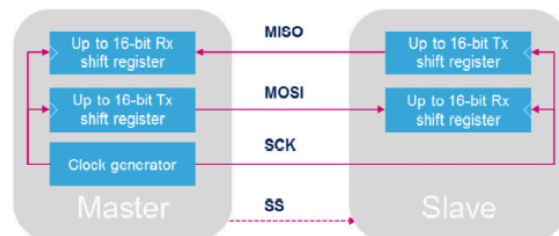


Interconnection of SPI nodes

6

Various master - slave interconnections are supported

- Master always provides clock and controls all the traffic (selects slave for communication)
- Data can be exchanged in both directions in parallel
- In Full-duplex mode (bidirectional), both master and slave transmit and receive data at the same time



What is SPI?

SPI is a fast and simple **serial communication protocol**. It lets your STM32G0 **talk to other chips** like sensors, memory chips, displays, etc.

♦ Why SPI?

- Only **a few wires** needed
- **Fast** and **full-duplex** (send & receive at same time)
- **Configurable** (data size, clock speed, etc.)
- Can connect to **many devices** using one master

SPI Basics – How it Works

SPI uses **4 main wires**:

1. **SCK** – Serial Clock (from master to sync data)
2. **MOSI** – Master Out, Slave In (master sends data)
3. **MISO** – Master In, Slave Out (slave sends data)
4. **NSS (or SS)** – Slave Select (activates a specific slave)

 **Major Point:** SPI needs **at least SCK + one data line (MOSI or MISO)** to work. NSS helps pick who the master is talking to.

SPI Modes

There are different communication **modes** depending on how you want to send/receive data:

♦ Full-Duplex

- Uses **MOSI + MISO**
- Data flows both ways at once

◆ Simplex

- Data only flows **one way**
- Only one data line used

◆ Half-Duplex

- **One wire** for both directions, like a **walkie-talkie**
- Data goes one way at a time
- You need to alternate who is sending/receiving

Why resistor is used in Half Duplex?

Ans: To provide a pull up to a specific voltage(eg 3.3V or 5V) on the communication line when the device is not transmitting.

Necessary because open drain outputs can only actively pull the line low, they don't actively pull it high. Without Pullup resistor there is no bidirectional communication.

🟡 **Major Point:** Full-duplex is fastest, half-duplex saves pins.

🧠 SPI Configurations

SPI is **highly configurable**:

- **Data size:** 4 to 16 bits
- **Order:** LSB first or MSB first
- **Clock Polarity & Phase (CPOL & CPHA):** How data is synced with clock
- **CRC Check:** Optional error detection
- **DMA Support:** Offload CPU when transferring data

🟡 **Major Point:** Adjust clock, data size, and timing to match the external device.



SPI Topologies (Network Arrangements)

1. Star Topology (Common)

- One master, multiple slaves
- Master picks which slave to talk to using **separate NSS pins**
- Flexible and easy to manage
- If no separate SS pin → data from multiple slaves **can mix and corrupt** on MISO line.
- If master gives HIGH to slave1 and LOW to slave2 at same time → data collides.

2. Circular Topology

- All devices connected in a **ring**
- One shared NSS line
- Data moves through all devices
- Less common and more complex

3. Multi-Master

- 2 masters take turns talking
- They **check each other's NSS** lines to avoid collision



Major Point: Star topology is most used in real-world SPI designs.



Data Handling with FIFOs and DMA

- SPI has **2 FIFOs** (Transmit & Receive), each **32-bit**
- **FIFO** = small buffer holding data temporarily

- Use **DMA** to auto-transfer data with less CPU load
- **Packing Mode** lets you write two 8-bit frames as one 16-bit access = faster

■ **Major Point:** Use DMA + FIFO + packing for **high-speed data** without stressing the CPU.

CRC for Data Integrity

- SPI supports **8 or 16-bit CRC**
- CRC sent at end of transmission
- Receiver checks CRC to see if data got corrupted

■ **Major Point:** CRC is useful when you're sending important or long data streams.

Slave Select (NSS) Pin

- NSS tells a slave when it should listen
- **Hardware or software controlled**
- In multi-master setup, it's crucial to avoid bus conflicts
- In **NSS pulse mode**, NSS goes high between frames (used in some advanced setups)

■ **Major Point:** Even in simple setups, using NSS avoids accidental miscommunication.



SPI Speed & Clock Considerations

- SPI clock speed must be $\leq$ **half of the bus frequency**
Max SPI speed = $PCLK \div 2$
- Use **prescaler** to slow down clock for slower devices
- Consider:
 - **Cable length**
 - **Number of nodes**
 - **GPIO speed setting**
 - **Temperature & voltage** (they affect signal timing)



Major Point: Always match your clock speed to your slowest device or physical setup.



Important Notes and Tips

- SPI can't work in **Stop/Standby/Shutdown modes**
- **Don't disable SPI** immediately after DMA or TX empty—check status first
- **Busy flag (BSY)** tells if SPI is still active (don't rely on just FIFO empty flags)
- In **transmit-only** mode, ignore RX data and overrun flags

SPI vs UART

 Feature	SPI (Serial Peripheral Interface)	UART (Universal Asynchronous Receiver/Transmitter)	
1. Wires Required	4 (MOSI, MISO, SCK, SS)	2 (TX, RX)	
2. Speed	Faster (up to tens of MHz)	Moderate (up to a few Mbps)	
3. Communication	Full-Duplex	Full-Duplex	
4. Clock	 Requires shared clock (SCK)	 No clock (asynchronous)	
5. Number of Devices	1 Master, Multiple Slaves	Only 2 devices (1-to-1)	
6. Addressing	 No addressing	 No addressing	
7. Data Checking	Optional CRC	Optional parity	
8. Use Case	Fast communication between ICs	Long-distance or serial PC-to-MCU communication	